

RentalOOP:
an Object Oriented Approach
for a Car Rental Platform

F415299
Institute of Digital Technologies

MSc - LLP109 Software Report

November, 2024

Contents

1 Introduction . . . 4
1.1 Overview of the Program . . . 4
1.2 Industrial Application . . . 4
1.3 Relation to Digital Technologies . . . 4
2 Program Structure . . . 5
2.1 Program Architecture . . . 5
2.1.1 Modules . . . 5
2.1.2 Class Interactions . . . 5
2.1.3 Workflow . . . 7
2.2 Coding and Documentation Standards . . . 7
3 Application Development . . . 8
3.1 Classes . . . 8
3.2 Dictionaries . . . 8
3.3 Control Flow . . . 9
3.4 Agile Methodology . . . 9
4 Program Functionality . . . 10
4.1 User Interaction . . . 10
4.2 Testing . . . 11
4.2.1 Test 1: renting an available car . . . 11
4.2.2 Test 2: view available vehicles . . . 12
4.2.3 Test 3: return a previously rented car . . . 12
4.2.4 Test 4: exit the rental platform . . . 13
4.2.5 Additional test: loyalty programme discount . . . 13
4.3 Known Issues and Attempted Solutions . . . 14
5 Conclusion . . . 14
5.1 Project Performance . . . 14
5.2 Recommendations for Future Development . . . 15
6 Acknowledgements . . . 15
References . . . 16
A Requirements Document . . . 17
A.1 Project Overview . . . 17
A.2 Functional Requirements . . . 17
A.2.1 Rent a Car . . . 17

A.2.2 View Available Cars . . . 17
A.2.3 Return a Car . . . 17
A.2.4 Exit Platform . . . 17
A.2.5 Loyalty Programme Discount . . . 17
A.3 Non-Functional Requirements . . . 18
A.3.1 Usability . . . 18
A.3.2 Performance . . . 18
A.3.3 Reliability . . . 18
A.4 Assumptions and Constraints . . . 18
A.4.1 Assumptions . . . 18
A.4.2 Constraints . . . 18
B User Guide . . . 19
B.1 Getting Started . . . 19
B.1.1 Run via Terminal . . . 19
B.1.2 Run via VSCode . . . 19
B.2 User Login . . . 19
B.3 Navigating the Platform . . . 19
B.3.1 View Shop's Available Cars . . . 19
B.3.2 Rent Cars . . . 19
B.3.3 Return a Car . . . 20
B.3.4 View Loyalty Programme . . . 20
B.4 Error Messages . . . 20
B.4.1 No Car (Type) Available . . . 20
B.4.2 Invalid Option . . . 20
B.4.3 No Car to Return . . . 20
B.4.4 Rent One Car per Customer . . . 20
B.5 Exit the Platform . . . 20

1. Introduction

1.1. Overview of the Program

The car rental platform is designed to provide users with a simple, interactive interface to manage car rentals through essential tasks, such as viewing available cars, renting and returning cars. The platform enables a customer to log into the rental system by entering their name, and performs various actions efficiently and intuitively. Built using Python and Object Oriented Programming (OOP) principles, the program organises its functionality into modular classes, including **Car**, **Customer**, and **RentalShop**, managed by a central controller file, **main.py**. Each class serves a distinct purpose in the system's operation, where: **Car** objects represent individual cars, holding their daily rates; **Customer** objects retain customer information; and **RentalShop** manages the inventory and rental transactions, enabling a seamless rental experience.

1.2. Industrial Application

The platform has strong potential for real-world applications, especially for small-to-medium car rental businesses seeking to streamline their rental processes. By automating the management of the car inventory, rentals and returns, this system can improve operational efficiency and reduce the chance of human error. Real-time updates on car availability and customer data facilitate smoother transactions, minimising wait times and enabling businesses to serve more customers effectively. The platform offers customers an organised and quick rental process, ensuring a positive experience and reducing friction typically associated with rental operations.

1.3. Relation to Digital Technologies

RentalOOP contributes meaningfully to Digital Technologies by demonstrating the effectiveness of OOP, Agile methodology, and modular programming in software projects. OOP principles enhance code organisation, readability, and reusability, ensuring the platform can be easily extended or modified. Despite being an individual project, adopting an Agile approach supports iterative development, allowing continuous testing and improvements. By structuring the code into distinct, reusable components, the project emphasises modularity and scalability - core aspects of modern digital technologies. Furthermore, the platform effectively addresses data management and backend processing, showcasing how digital solutions can optimise both customer interaction and operational efficiency in a scalable manner.

2. Program Structure

2.1. Program Architecture

The program is structured into four modular files, each handling a specific aspect of the platform. the organisation of these files reflects an OOP design, with classes that encapsulate data and behaviours for cars, customers, and rental transactions. The platform's structure supports scalability, as each module is self-contained and interactively organised within the main program file.

2.1.1. Modules

- **main.py**: Acts as the program's entry point, managing the flow and handling the user's inputs. This file initialises the **RentalShop** with the name *RentACar* and a stock of cars (4 hatchbacks, 3 sedans and 3 SUVs) and produces a **Customer** profile with the name entered by the user, enabling them to immediately start using the platform.
- **cars.py**: Contains the **Car** parent class and its child classes for the different car models: **Hatchback**, **Sedan**, and **SUV**. Each child class overwrites the parent's attributes for the specific daily rental rates, based on the length of time and customer membership.
- **customer.py**: Implements the **Customer** class, managing customer-specific details and membership status in the loyalty program. The **VIP** class inherits the details from the **Customer** class, however, it overwrites the membership status to *True*, stating that the customer has signed up for the loyalty programme.
- **rental_shop.py**: Defines the **RentalShop** class, which is responsible for managing the car inventory, handling rental and return transactions, calculating rental bills and signing the customer in to or out of the loyalty programme.

2.1.2. Class Interactions

Figure 1 illustrates the relationship among classes in the rental system, emphasising key OOP principles such as aggregation, composition and inheritance, using a Unified Modelling Language (UML) diagram.

The **RentalShop** class employs composition to manage **Car** instances, represented by a black diamond in the UML diagram. This indicates that **Car** objects are created and exist solely within the context of **RentalShop**, establishing a one-to-many relationship.

Aggregation is depicted between **RentalShop** and **Customer** classes using a white diamond, representing a one-to-one association, where each shop instance is linked to a single customer at

any given time. For simplicity, the **Customer** class allows only one car rental per instance, ensuring flexibility in managing rentals.

The **Customer** class includes a subclass, **VIP**, which inherits all attributes and methods from its parent while overriding the **is_vip** method and introducing a **transfer_customer_details** method. Inheritance is denoted by a white triangle in the UML diagram, which also highlights the **Car** class having three specialised subclasses: **Hatchback**, **Sedan**, and **SUV**.

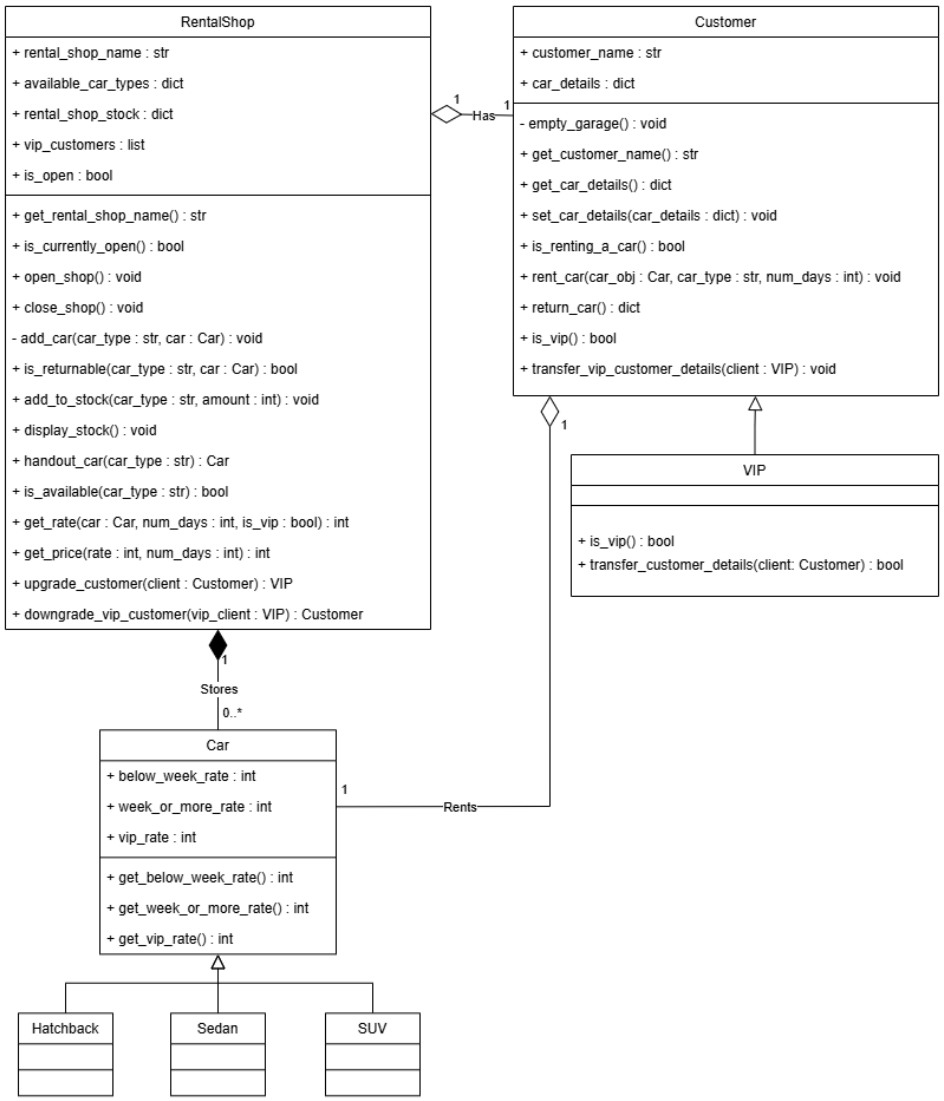


Fig 1: UML Class Diagram.

2.1.3. Workflow

The **main.py** script initialises a **RentalShop** instance, populating its inventory with four hatchbacks, three sedans, and three SUVs using the **add_to_stock** method. A welcome message is then displayed to the user via the command line. A **Customer** instance is created upon receiving the user's name, which is directly assigned as the **customer_name** attribute without validation. For simplicity, user authentication is limited to name entry. Subsequently, the user interacts with the system through a menu offering options to rent or return cars, view available stock, join or leave the loyalty programme, or exit the platform.

Interactions between the user and the shop are managed through **Customer** and **RentalShop** methods. To rent a car, the shop validates availability and, if successful, provides a **Car** instance of the requested type using the **handout_car** method. This instance is then stored by the **Customer** object through the **rent_car** method. For car returns, the **return_car** method in the **Customer** class provides details to the shop, which verifies the returnable status using its **is_returnable** method, ensuring that the car type matches its inventory. Upon validation, the shop re-adds the car to its inventory using a private **add_car** method and calculates the bill via the **get_price** method.

Customers can upgrade to a VIP status or revert to regular status through the shop's **upgrade_customer** and **downgrade_customer** methods. When upgrading, a **VIP** instance is created, transferring all details from the **Customer** instance using the **transfer_customer_details** method. The reverse process replaces the **VIP** instance with a **Customer** instance, retaining all attributes.

2.2. Coding and Documentation Standards

The platform strictly follows Python's PEP 8 coding standards (Van Rossum 2013) to ensure consistency and readability. Docstrings are included in all classes and methods to describe their purpose, parameters, and return values, while inline comments clarify complex logic. Naming conventions are consistently applied: attributes, methods and modules use lowercase with underscores separating words, while class names follow PascalCase. Indentation adheres to PEP 8 standards as well, using four spaces per level (Finer 2024). This compliance enhances collaborative development and ensures a clean, maintainable, and scalable codebase.

3. Application Development

3.1. Classes

RentalOOP heavily applies Object-Oriented Programming (OOP) principles to ensure a modular, maintainable, and scalable car rental platform. The primary classes include **RentalShop**, **Customer**, and **Car**, each designed to encapsulate related data and behaviours.

Data and methods related to car rentals are encapsulated within specific classes. For example, the **Customer** class contains attributes, such as *customer_name* and *car_details*, to store the customer's information, and methods to manage said information, including **get_customer_name** and **rent_car** - where the customer receives the car from the rental shop and stores its details. This encapsulation prevents data from being accidentally modified outside the class, enhancing reliability.

Polymorphism ensures flexibility and adaptability in the code. For instance, the **Customer** class has a child class, **VIP**, which overrides the **is_vip** method to offer unique functionality. This allows the platform to seamlessly manage different types of customers without duplicating code or altering the parent class, showcasing how polymorphism improves adaptability.

By adhering to these OOP principles, the platform achieves a clean and scalable design. Alternative approaches, such as procedural programming, would have resulted in less organised code, reduced reusability, and difficulty maintaining or scaling the system.

3.2. Dictionaries

Dictionaries are efficient key-value data structures, enabling fast lookups and updates. Their direct key-based access makes them ideal for applications requiring structured and quick data retrieval.

In the **RentalShop** class, dictionaries are meployed to efficiently manage car inventory through two key implementations: *available_car_types* and *rental_shop_stock*. The *available_car_types* dictionary maps car types (*Hatchback*, *Sedan*, *SUV*) as keys to their respective class definitions, enabling dynamic instantiation of specific car objects as needed to populate the shop's inventory.

The *rental_shop_stock* dictionary, on the other hand, organises actual car objects by type, facilitating quick inventory checks and efficient updates to car availability. For instance, when a car is rented or returned, the dictionary allows instant identification and modification of the relevant inventory data.

Lists and sets were deemed unsuitable alternatives on their own. Lists, while ordered, lack efficient key-based access, requiring linear searches. Sets, though fast, do not support key-value mappings. Dictionaries were selected for their structured, dynamic, and efficient data handling.

3.3. Control Flow

Control flows in the platform ensure smooth user interactions and logical system responses.
A while loop is used as the main program loop in **main.py**, ensuring that the platform remains active until the user chooses to exit. Within this loop, the user can repeatedly perform actions, such as viewing available cars, renting, or returning cars, without restarting the program. This design enhances user convenience and reflects real-world usage patterns.
Alternative approaches, such as recursion or multiple stand-alone loops, would lack the simplicity and efficiency of the current design. Recursion introduces unnecessary complexity, while stand-alone loops for each action could disrupt the user experience by requiring frequent program restarts.

3.4. Agile Methodology

RentalOOP was developed using an Agile methodology, emphasising iterative progress. Each feature - such as car rental, returns, or loyalty program integration - was designed, implemented, tested and documented sequentially before moving to the next. This approach ensured focused development and continuous improvement, suited for an individual project.

4. Program Functionality

4.1. User Interaction

RentalOOP operates through a command-line interface (CLI), providing users with a simple and accessible text-based system. The interaction begins when the user executes **main.py** through an IDE such as VSCode. The program prompts the user to log in by entering their name, and initialising their profile within the system.

The CLI is intuitive, offering prompts and feedback for each action. Fig. 2 provides a flowchart outlining the procedural steps and user decision points, illustrating how each action leads to an outcome.

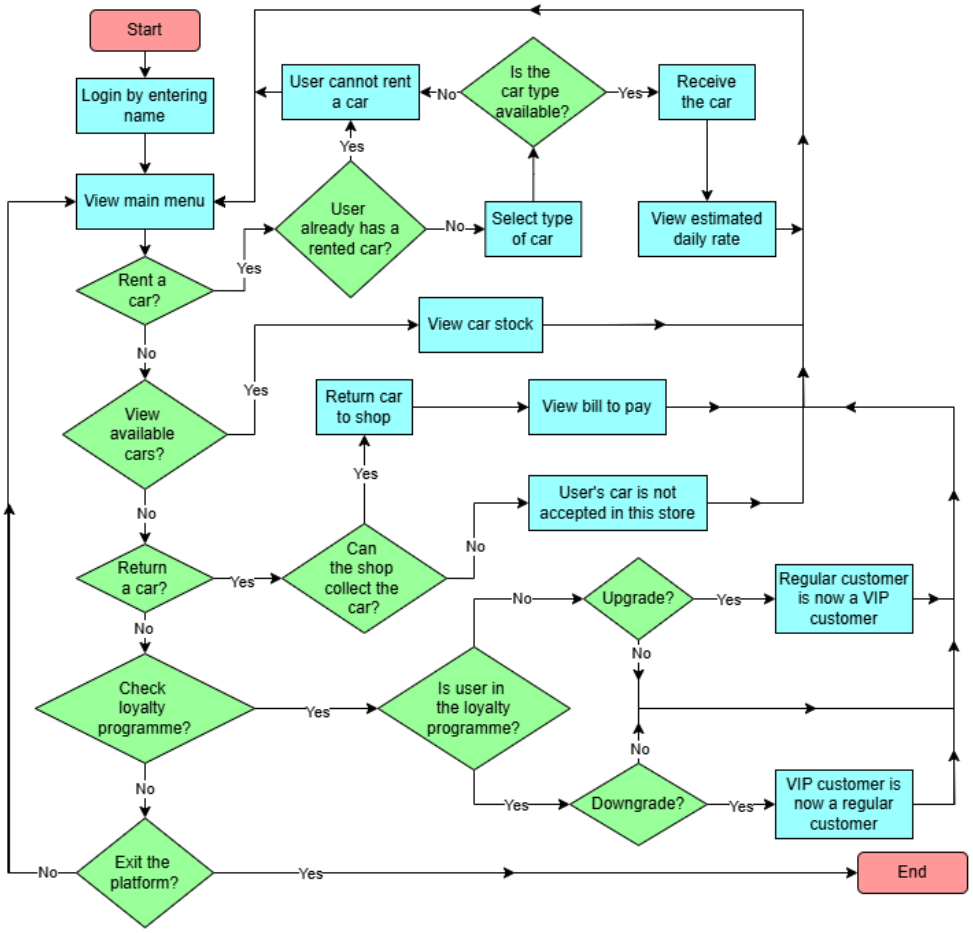
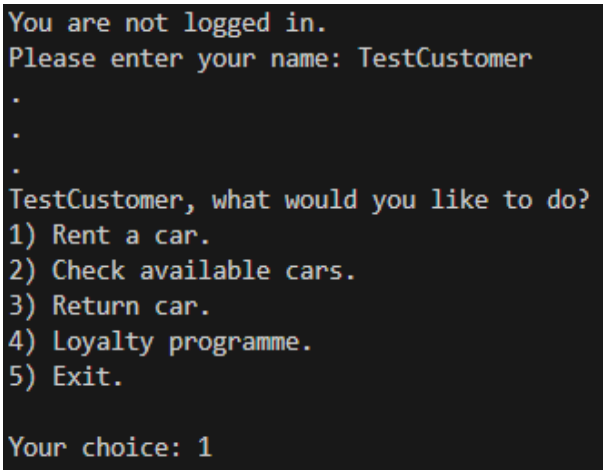


Fig 2: User Flow Chart. Diamonds are questions. Rectangles are actions.

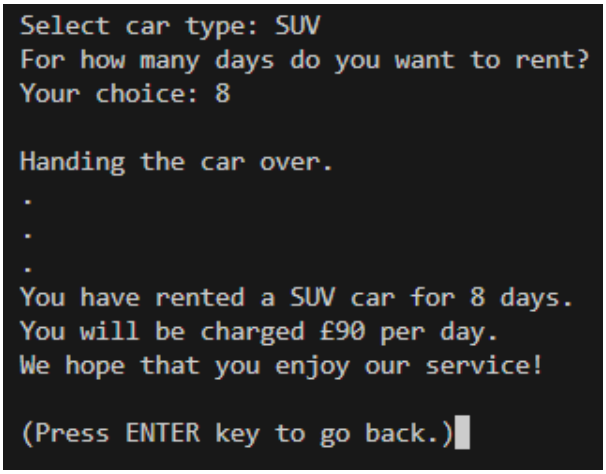
4.2. Testing

The platform was tested against the primary requirements outlined in the requirements document in Appendix A. These tests confirm the successful execution of core functionalities.

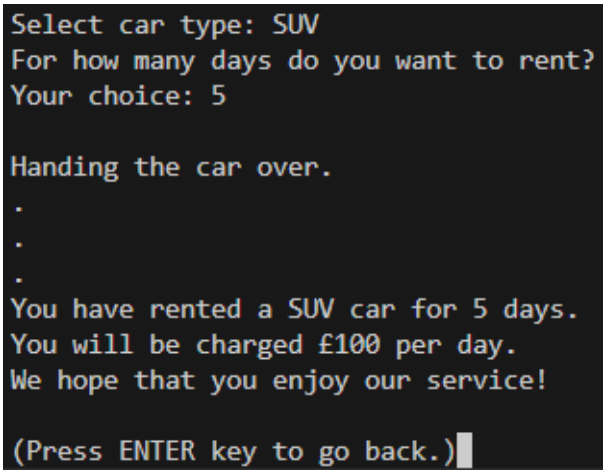
4.2.1. Test 1: renting an available car



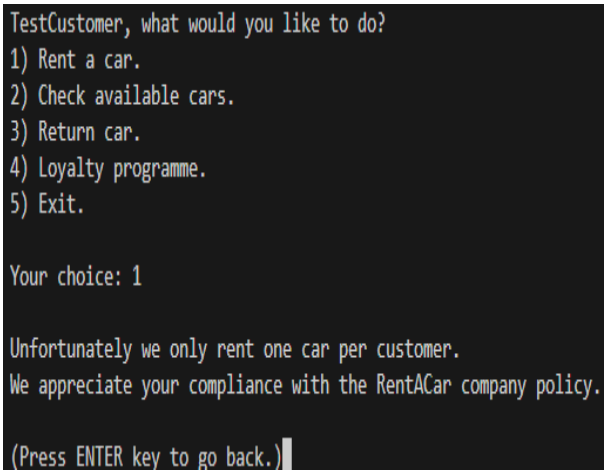
(a) Main menu.



(b) Renting a car for a week or longer.



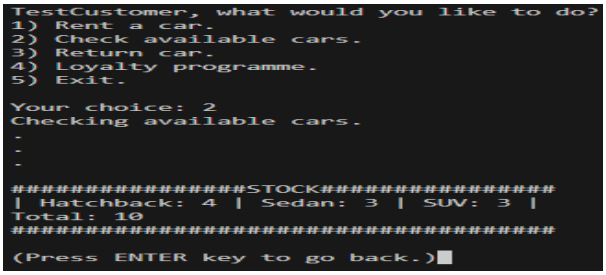
(c) Renting a car for less than a week.



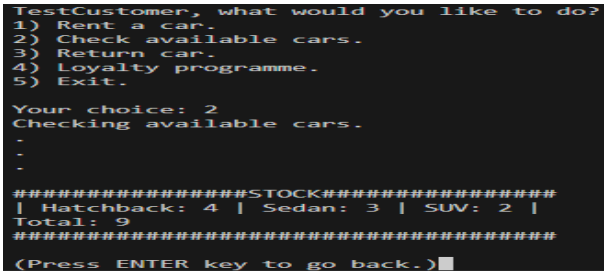
(d) Renting a car when currently renting one.

Fig 3: Test for renting an available car.

4.2.2. Test 2: view available vehicles



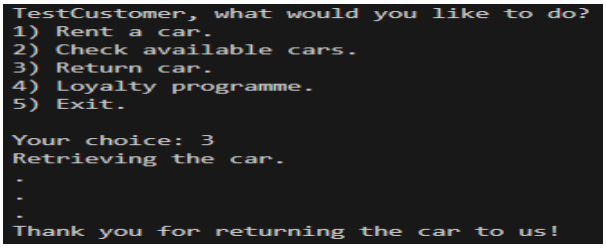
(a) The rental shop's initial stock.



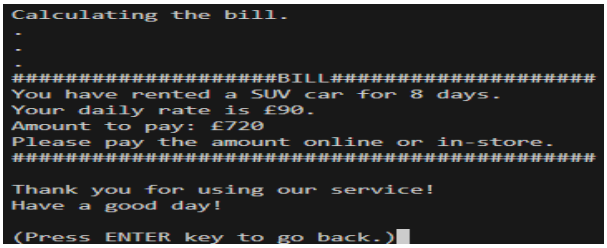
(b) View stock after a car has been rented.

Fig 4: Test for viewing all available cars.

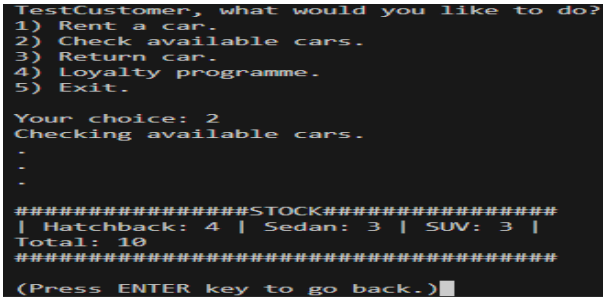
4.2.3. Test 3: return a previously rented car



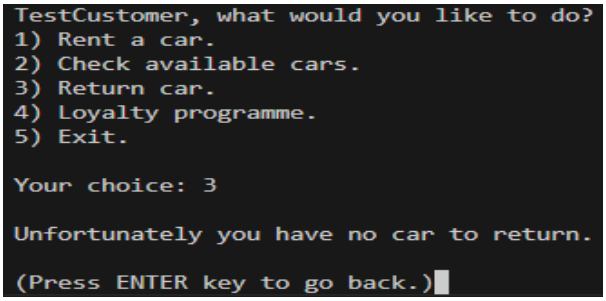
(a) User returns the rented car.



(b) Displaying the bill to pay.



(c) Stock is updated with the returned car.



(d) Returning a car when the user has none.

Fig 5: Test for returning a previously rented car.

4.2.4. Test 4: exit the rental platform

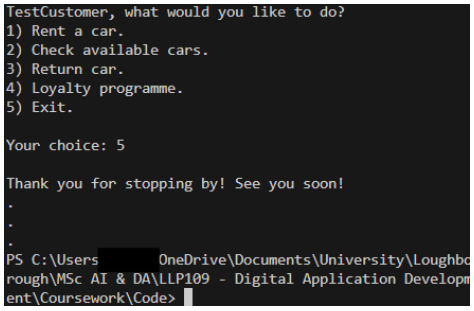
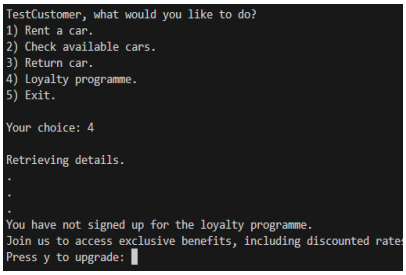
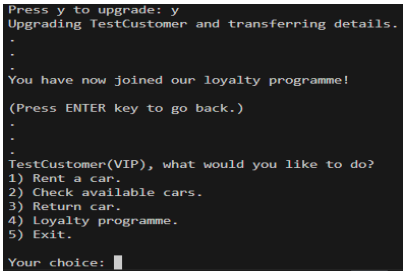


Fig 6: Test to exit the rental platform.

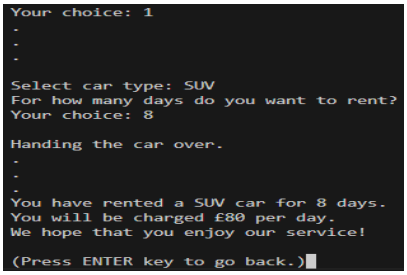
4.2.5. Additional test: loyalty programme discount



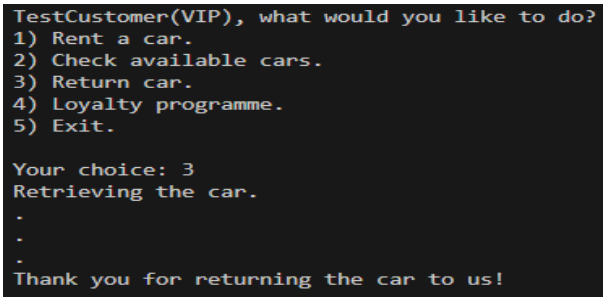
(a) Check if the user is a VIP.



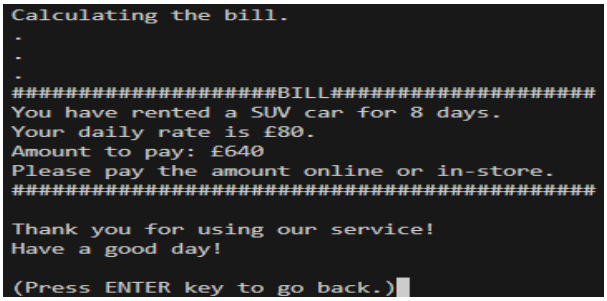
(b) Upgrade from regular to VIP.



(c) VIP customer rents a car.



(d) VIP customer returns their rented car.



(e) The bill for the VIP customer's rental.

Fig 7: Test for customer joining the shop's loyalty programme.

4.3. Known Issues and Attempted Solutions

Despite successful test results, the platform has limitations typical of a Minimum Viable Product (MVP).
Firstly, the platform supports only one active user at a time, assuming the user is always new, starts without a rented car, and has not signed up for the rental shop's loyalty programme. This simplification avoids potential errors such as conflicting rental statuses but reduces scalability for multiple users. Future iterations will include support for multiple users and session tracking.
The next issue concerns the user being able to rent a car as a regular customer, then join the loyalty program, and return the car at a discounted rate. This behaviour is inconsistent with typical business logic, as the price should reflect the user's status at the time of rental. Currently, for the MVP, the final bill is calculated based on the user's current status (regular or VIP) at the time of return. Implementing historical pricing based on the initial rental conditions would require more complex data tracking, which is planned for future development.
These challenges highlight the trade-offs made to deliver an MVP while maintaining functionality and user experience. Future enhancements will address these limitations, aligning the platform closer to real-world scenarios.

5. Conclusion

5.1. Project Performance

The car rental platform successfully meets and exceeds the specified requirements, demonstrating robust functionality within its scope. Key achievements include the application of Object-Oriented Programming (OOP) principles to create a modular and scalable system. The encapsulation of data and clear class hierarchies enhance code reusability and maintainability, confirming the suitability of the OOP approach for this project.
However, the platform has limitations that are characteristic of a Minimum Viable Product (MVP). While sufficient for early development, the reliance on a Command-Line Interface (CLI) limits user accessibility and usability. Additionally, the lack of persistent data storage - currently handled by in-memory structures like dictionaries - restricts long-term scalability and data security. Another limitation is the simplistic login mechanism, which only requests a user's name without proper authentication.

5.2. Recommendations for Future Development

Multiple enhancements are recommended to transition the platform from an MVP to a real-world application.
User experience (UX) and interaction may be improved by introducing a more user-friendly interface using tools like **tkinter** for a desktop Graphical User Interface (GUI) or **Flask** for a web-based solution.
In-memory data structures should be replaced with a robust database system such as PostgreSQL, allowing for secure, persistent storage of customer data, car inventories and transaction histories.
Multiple users and concurrent sessions can be supported by implementing a comprehensive login and registration system requiring user credentials like email and password.
Lastly, a detailed error-handling and logging mechanism should be developed to manage user inputs and unexpected behaviours, improving system reliability.
These enhancements would significantly improve the platform's robustness, usability, and scalability, making it a viable tool for real-world car rental businesses.

6. Acknowledgements

I extend my gratitude to Courtney Reed, the module leader, for her guidance in Object-Oriented Programming and Python principles foundational for RentalOOP. I also thank my peers and external testers for their feedback, especially on improving diagram clarity and readability, which greatly enhanced this work.

References

Finer, J. (Feb. 2024). *How to Write Beautiful Python Code With PEP 8 [online]*. Real Python. URL: https://realpython.com/python-pep8/#tabs-vs-spaces.

Van Rossum, G. (Aug. 2013). *PEP 8 – Style Guide for Python Code [online]*. Python Enhancement Proposals (PEPs). URL: https://peps.python.org/pep-0008/#code-lay-out.

Appendix A: Requirements Document

A.1. Project Overview

A Command-Line Interface (CLI) based car rental platform enabling users to view, rent, and return cars. Developed using Python and OOP principles to ensure modularity, scalability, and maintainability, with Agile methodology guiding iterative development.

A.2. Functional Requirements

Core features and capabilities necessary to meet user needs.

A.2.1. Rent a Car

The user can rent a hatchback, sedan, or SUV for a specified duration, receiving an estimated daily rate if the car is available.

A.2.2. View Available Cars

The system displays the count of available cars by type and the total inventory.

A.2.3. Return a Car

The user can return the rented car, updating the shop's stock for future rentals.

A.2.4. Exit Platform

The user can gracefully exit the platform.

A.2.5. Loyalty Programme Discount

The user can join or leave the shop's loyalty programme to access discounted rates.

A.3. Non-Functional Requirements

System attributes ensuring performance, usability, and reliability, but do not involve specific functionality.

A.3.1. Usability

The CLI must be intuitive and easy to navigate.

A.3.2. Performance

Operations like renting, returning and viewing must execute within 10 seconds.

A.3.3. Reliability

The system must maintain accurate data consistency, e.g. car availability.

A.4. Assumptions and Constraints

A.4.1. Assumptions

The user interacts with the platform through a CLI for basic rental operations.

A.4.2. Constraints

The Minimum Viable Product (MVP) relies on a CLI with no database integration, using in-memory storage only.

Appendix B: User Guide

This guide provides clear and concise instructions for using the platform via the Command-Line Interface (CLI) on Terminal or Visual Studio Code (VSCode). Follow the steps below to get started and navigate the platform.

B.1. Getting Started

Ensure Python is installed on your system.
Download and extract the project files to a folder.

B.1.1. Run via Terminal

Open the Terminal application and navigate to the project folder, then start the platform by running:

```
python main.py
```

B.1.2. Run via VSCode

Open VScode and load the project folder.
Open **main.py**.
Click the **Run** button or press **F5** to start the program.

B.2. User Login

When prompted, enter your name to sign in.

B.3. Navigating the Platform

B.3.1. View Shop's Available Cars

Enter **2** as your option in the main menu to display all available cars for renting.

B.3.2. Rent Cars

Enter **1** as your option in the main menu, then choose the type of car (Hatchback/Sedan/SUV), and then the number of days to rent.

B.3.3. Return a Car

Enter **3** in the menu to return your rented car.

B.3.4. View Loyalty Programme

Enter **4** to manage loyalty programme membership.
If signed in, enter **y** to sign out, lose the *(VIP)* tag, and forfeit discounted rates.
If not signed in, enter **y** to sign in and access discounted rental rates.

B.4. Error Messages

B.4.1. No Car (Type) Available

Choose another type of car, if the selected type is unavailable.
Select **2** in the main menu to view the available cars before renting.

B.4.2. Invalid Option

Enter a number between 1-5 or command as shown in this guide.

B.4.3. No Car to Return

Rent a car first before deciding to return it.

B.4.4. Rent One Car per Customer

Unfortunately, you can only rent one car through this platform.
To rent another car, return your currently rented one first.

B.5. Exit the Platform

To safely exit, select option **5** in the main menu.